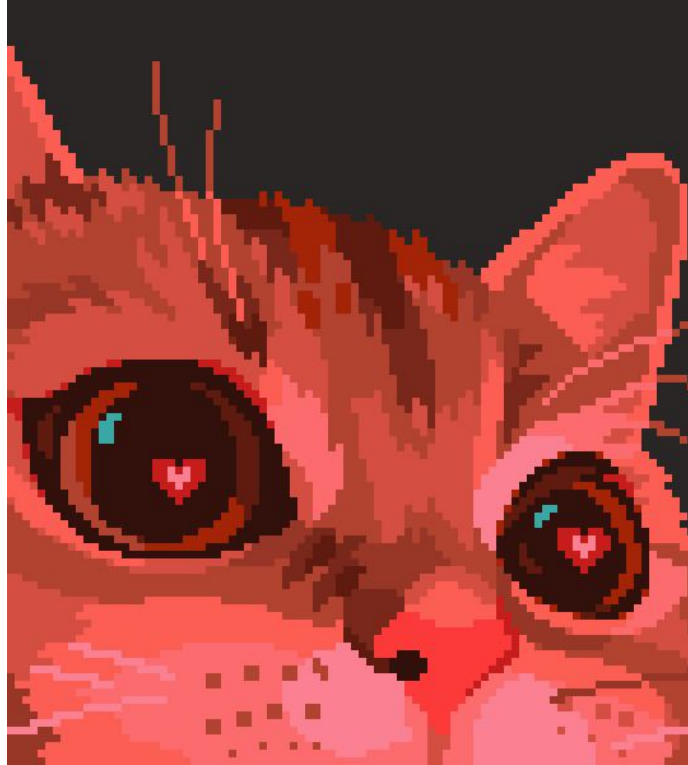




TSWAP Report

Aayush Suthar

June 25, 2026

Prepared by: Aayush Suthar

Table of Contents

- Table of Contents
- Disclaimer
- Risk Classification
- Protocol Summary
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees
 - * [H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens
 - * [H-3] `TSwapPool::sellPoolToken` mismatches input and output tokens causing the users to receive the incorrect amount of tokens
 - * [H-4] In `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invariant of $x * y = k$
 - Medium
 - * [M-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline
 - Low
 - * [L-1] `TSwapPool::LiquidityAdded` event has parameters out of order
 - * [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given
 - Informational
 - * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed
 - * [I-2] Lacking zero address checks
 - * [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`

Disclaimer

The auditor makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this

document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Protocol Summary

Audit Details

The findings described in this document corresponds the following commit hash:

e643a8d4c2c802490976b538dd009b351b1c8dda

Scope

```
./src/  
|__PoolFactory.sol  
|__TSwapPool.sol
```

Roles

1. Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
2. Users: Users who want to swap tokens.

Executive Summary

Executive Summary

This audit was conducted as a guided learning exercise through the Cyfrin Updraft Smart Contract Security course. The TSWAP protocol was reviewed as

part of the course curriculum to develop practical auditing skills and familiarity with the audit process end to end.

The review was conducted by Aayush Suthar following the methodology taught in the course, including manual code review, static analysis, and proof of concept development using Foundry.

Despite being a learning exercise, the findings documented in this report represent real vulnerabilities present in the codebase. Two High severity issues and one Informational finding were identified during the review.

Tools used: Foundry, Manual Review, Slither, Aderyn

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	2
Info	3
Total	10

Findings

High

[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users

Recommended Mitigation:

```
function getInputAmountBasedOnOutput(
    uint256 outputAmount,
    uint256 inputReserves,
    uint256 outputReserves
)
    public
    pure
    revertIfZero(outputAmount)
    revertIfZero(outputReserves)
    returns (uint256 inputAmount)
{
-     return ((inputReserves * outputAmount) * 10000) / ((outputReserves - outputAmount) * 10000);
+     return ((inputReserves * outputAmount) * 1000) / ((outputReserves - outputAmount) * 1000);
}
```

[H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens

Description: The `swapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done in

`TSwapPool::swapExactInput`, where the function specifies a `maxInputAmount`.

Impact: If market conditions change before the transactions process, the user could get a much worse swap.

Proof of Concept: 1. The price of WETH right now is 1000USDC 2. User inputs a `swapExactOutput` looking for 1 WETH. 1. `inputToken = USDC` 2. `outputToken = WETH` 3. `outputAmount = 1` 4. `deadline = whatever` 3. The function does not offer a `maxInputAmount` 4. As transaction is pending in the mempool, the market changes! and the price moves HUGE -> 1 WETH is not 10,000 USDC. 10x more than the user expected. 5. The transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC.

Recommended Mitigation: We should include a `maxInputAmount` so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol.

```
function swapExactOutput(  
    IERC20 inputToken,  
+    uint256 maxInputAmount,  
+  
+  
+  
+    inputAmount = getInputAmountBasedOnOutput(outputAmount, inputReserves, outputReserve  
+    if(inputAmount > maxInputAmount){  
+        revert();  
+    }  
+    _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-3] TSwapPool::sellPoolToken mismatches input and output tokens causing the users to receive the incorrect amount of tokens

Description: The `sellPoolToken` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapper amount.

This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: User will swap the wrong amount of tokens, which is a server disruption of protocol functionality

Recommended Mitigation: Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolToken` function to accept a new parameter (i.e., `minWethToReceive` to be passed to `swapExactInput`)

```

function sellPoolTokens(uint256 poolTokenAmount) external returns (uint256 wethAmount,
+   uint256 minWethToReceive
) {
-   return swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount, uint64(block.timestamp));
+   return swapExactOutput(i_poolToken, poolTokenAmount, i_wethToken, minWethToReceive);
}

```

Additionally, it might be wise to add a deadline to the function, as there is currently no deadline.

[H-4] In TSwapPool::_swap the extra tokens given to users after every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follows a strict invariant of $x * y = k$. Where: -
x: The balance of the pool token - y: The balance of WETH - k: The constant product of the two balance

This means, that whenever the balances change in the protocol, the ration between the two amounts should remain constant, hence the k. However, this is broken due to extra incentive in the _swap function. Meaning that over time the protocol funds will be drained.

The following block of code is responsible for the issue

```

swap_count++;
if (swap_count >= SWAP_COUNT_MAX) {
    swap_count = 0;
    outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000);
}

```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put, the porotocol's core invariant proken

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of 1e18 tokens 2. The user continues to swap untill all the protocol funds are drained

Proof Of Code Place the following into TSwapPool.t.sol.

```

function testInvairantBorkenProof() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 outputWeth = 1e17;
}

```

```

vm.startPrank(user);
poolToken.approve(address(pool), type(uint256).max);
poolToken.mint(user, 100e18);
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));

int256 startingY = int256(weth.balanceOf(address(pool)));
int256 expectedDeltaY = int256(-1) * int256(outputWeth);

pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
vm.stopPrank();

uint256 endingY = weth.balanceOf(address(pool));
int256 actualDeltaY = int256(endingY) - int256(startingY);

assertEq(actualDeltaY, expectedDeltaY);
}

```

Recommended Mitigation: Remove the extra incentives. If you want to keep this in, we should account for the change in the $x * y = k$ protocol invariants. Or, we should set aside tokens in the same way we do with fees.

Medium

[M-1] TSwapPool::deposit is missing deadline check causing transactions to complete even after the deadline

Description: The deposit function accepts a deadline parameter, which according to the documentation is “/// @param deadline The deadline for the transaction to be completed by”. However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavourable.

Impact: Transaction could be send when market condition are unfavourable to deposit, even when adding a deadling parameter

Proof of Concept: The deadline parameter is unused.

Recommended Mitigation: Consider making the following change to the functions.


```

function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline
)
    external
+   revertIfDeadlinePassed(deadline)
    revertIfZero(wethToDeposit)
    returns (uint256 liquidityTokensToMint)
{

```

Low

[L-1] TSwapPool::LiquidityAdded event has parameters out of order

Description: When the `LiquidityAdded` event is emitted in the `TSwapPool::_auditLiquidityMintAndTrans` function, it logs values in an incorrect order. The `poolTokenToDeposit` value should go in the third parameter position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain functions potentially malfunctioning.

Recommended Mitigation:

```

-   emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+   emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);

```

[L-2] Default value returned by TSwapPool::swapExactInput results in incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the names return value `output` it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Recommended Mitigation:

```

    returns (uint256 output)
{
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

-   uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);
+   uint256 output = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);
}

```

```

-         if (outputAmount < minOutputAmount) {
-             revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
-         }
+         if (output < minOutputAmount) {
+             revert TSwapPool__OutputTooLow(output, minOutputAmount);
+         }

-         _swap(inputToken, inputAmount, outputToken, outputAmount);
+         _swap(inputToken, inputAmount, outputToken, output);
    }

```

Informational

[I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed

```

- error PoolFactory__PoolDoesNotExist(address tokenAddress);

```

[I-2] Lacking zero address checks

```

    constructor(address wethToken) {
+         if(wethToken == address(0)){
+             revert();
+         }
        i_wethToken = wethToken;
    }

```

[I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`

```

-         string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).name());
+         string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).symbol());

```